Indice

Interazioni con il sistema.....2
Specifiche casi d'uso.....2
Casi d'uso del paziente.....3
Casi d'uso del diabetologo.....8
Diagrammi di attività.....14
***Sviluppo e progettazione*.....17**
Note sul processo di sviluppo.....17
Progettazione e pattern architetturali.....18
Implementazione e design pattern.....21
Dao.....21
Facade.....21
***Attività di test e validazione*.....23**
Unit test.....24
Test degli sviluppatori.....25
Test utente generico.....27

Interazioni con il sistema

Specifiche casi d'uso

Note generali

Il sistema informatico progettato costituisce un servizio clinico per la gestione di pazienti diabetici, in particolare diabete di tipo 2. Sono presenti due tipologie di attori: paziente e diabetologo, entrambi accedono al sistema tramite autenticazione. In caso questa vada a buon fine, entrambi vengono reindirizzati alla rispettiva schermata iniziale.

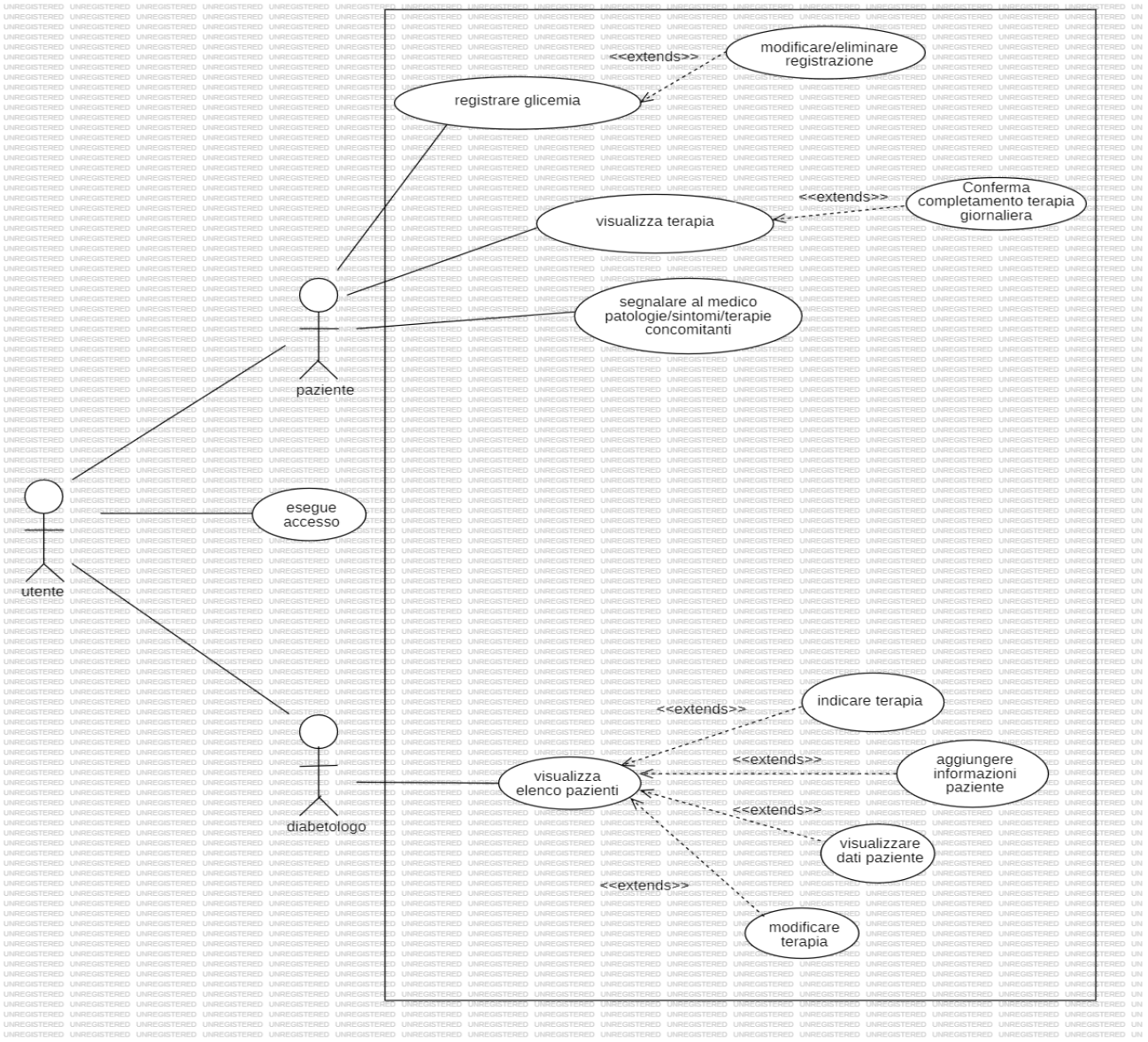

Figure 1: Casi d'uso

Casi d’uso del paziente

REGISTRARE IL LIVELLO DI GLICEMIA

Il paziente può registrare, modificare o eliminare le misurazioni giornaliere di glicemia prima e dopo i pasti**,** nel caso la rilevazione fosse fuori i parametri viene segnalato il problema al paziente tramite un alert.

Attori: Paziente
Precondizioni: Il paziente è autenticato

Passi:

1. Il paziente accede al sistema
2. Il paziente è introdotto all’interfaccia di base
3.
    - 3.1. Il paziente accede all’interfaccia delle proprie misurazioni
        - 3.1.1. Il paziente modifica o elimina la misurazione
    - 3.2. Il paziente accede all’interfaccia di inserimento delle misurazioni
        - 3.2.1. Il paziente inserisce la misurazione

Postcondizioni: Misurazione registrata, modificata o eliminata dal sistema

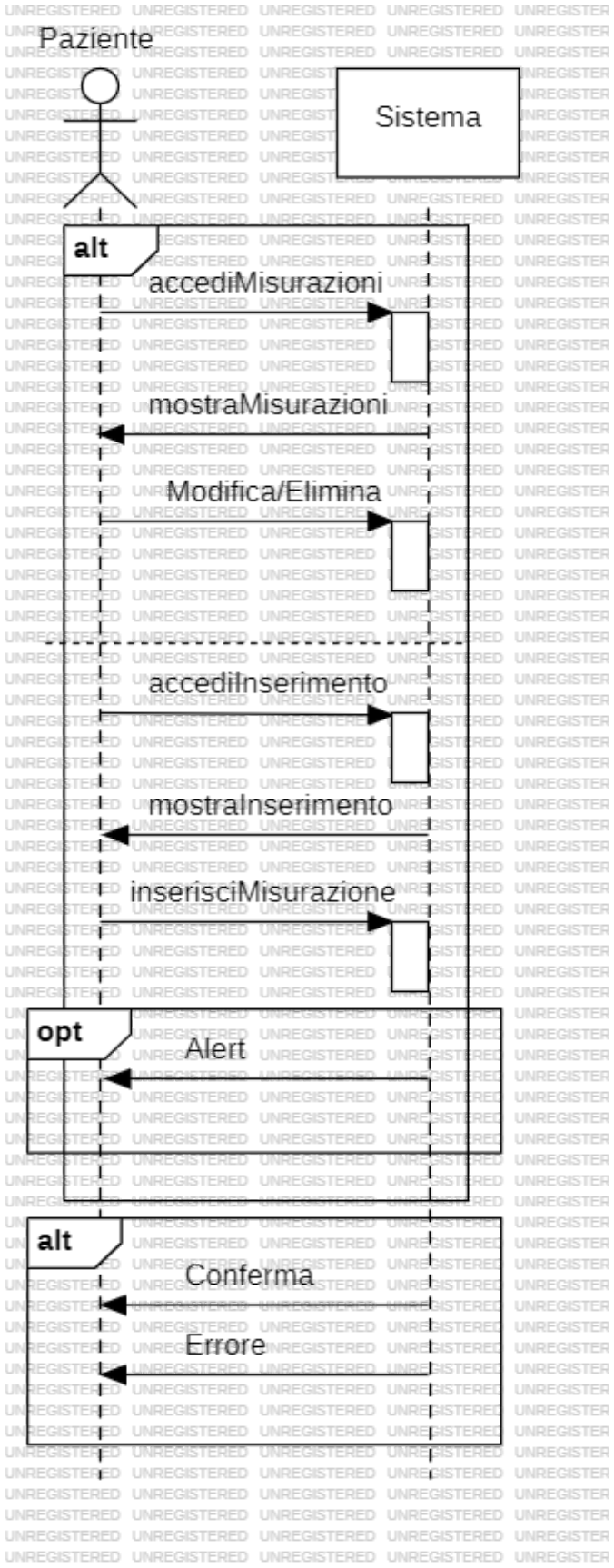

Figure 2: Registra livello glicemia

SEGNALAZIONE DI UN SINTOMO/TERAPIA/PATOLOGIA

Un paziente può registrare un sintomo, una terapia e/o una patologia, indicando la data di riferimento. Per semplificare l'inserimento, sono disponibili alcuni sintomi generali predefiniti tra cui scegliere, con la possibilità di aggiungerne di nuovi manualmente. È inoltre presente un campo di testo dedicato, in cui il paziente può descrivere eventuali patologie o terapie concomitanti.

Attori: Paziente
Precondizioni: Il paziente è autenticato, ha un medico assegnato e una terapia

Passi:

1. Il paziente accede al sistema
2. Il paziente è introdotto all'interfaccia di base
3. Il paziente accede all'interfaccia per inserire sintomi e/o terapie e patologie
4. Il paziente inserisce i sintomi da segnalare terapie e/o patologie
5. Se il paziente presenta patologie e/o terapie concomitanti le inserisce nell'apposito campo

Postcondizioni: La segnalazione è aggiunta al sistema

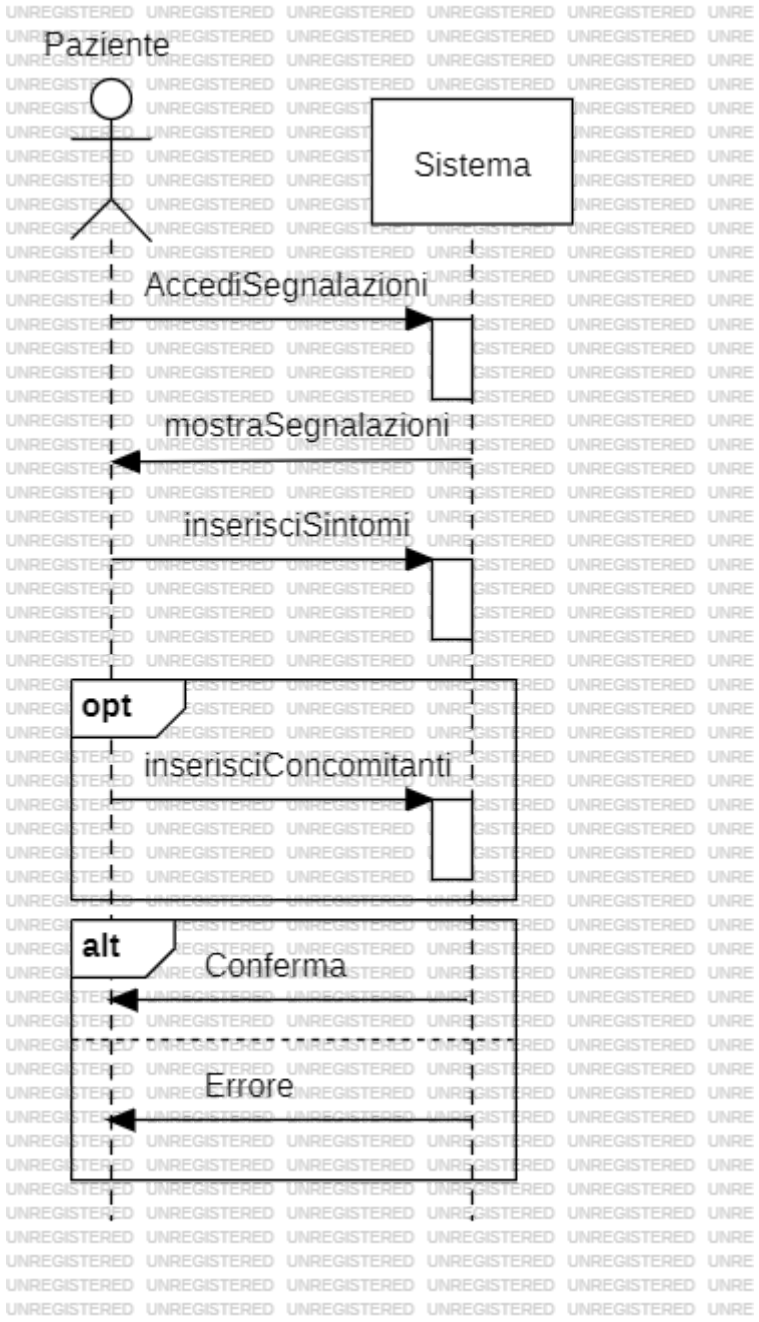

Figure 3: Segnalare sintomo

REGISTRARE L'ASSUNZIONE DI FARMACI

Un paziente può registrare l'assunzione di un farmaco prescritto dal diabetologo, specificando: data, farmaco e dose.

Attori: Paziente
Precondizioni: Il paziente è autenticato, ha un medico assegnato e una terapia

Passi:

1. Il paziente accede al sistema
2. Il paziente è introdotto all'interfaccia di base
3. Il paziente accede all'interfaccia per registrare l'assunzione di un farmaco
4. Il paziente registra l'assunzione di uno o più farmaci

Postcondizioni: L'assunzione del farmaco è stata aggiunta al sistema

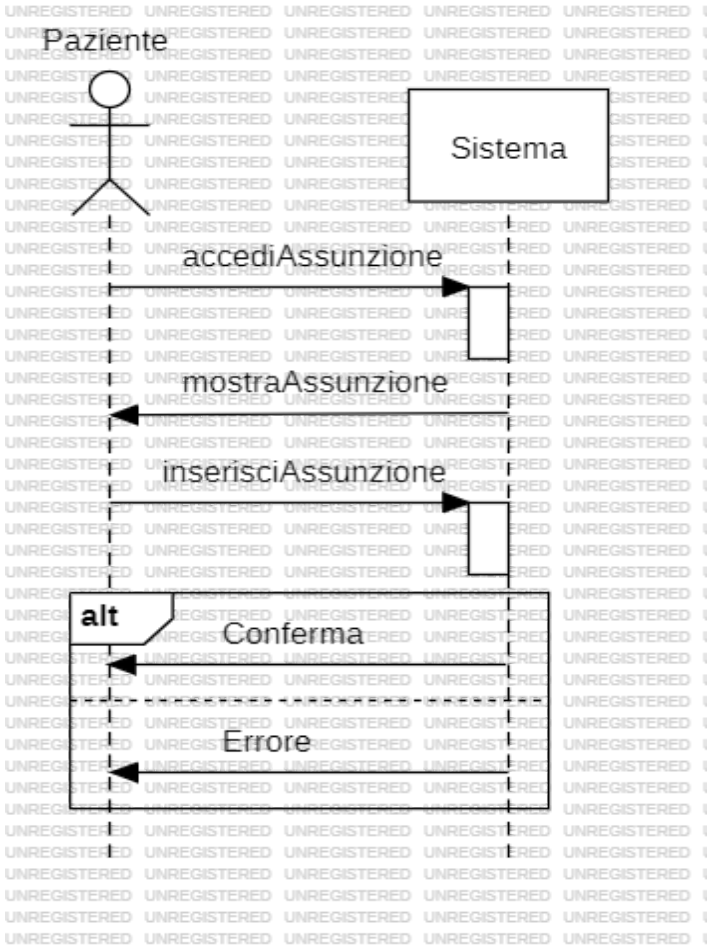

Figure 4: Registra assunzione

Casi d’uso del diabetologo

INDICARE LA TERAPIA DEI PAZIENTI

Un diabetologo può specificare la terapia che il paziente deve seguire, specificando: farmaco, numero di assunzioni giornaliere, quantità per assunzione ed eventuali indicazioni.

Attori: Diabetologo

Precondizioni: Il diabetologo è autenticato e ha dei pazienti assegnati

Passi:

1. Il diabetologo accede al sistema
2. Il diabetologo è introdotto all’interfaccia di base
3. Il diabetologo sceglie il paziente in cura
4. Il diabetologo accede all’interfaccia per l’aggiunta di una terapia
5. Il diabetologo indica la terapia

Postcondizioni: La terapia indicata è aggiunta al sistema

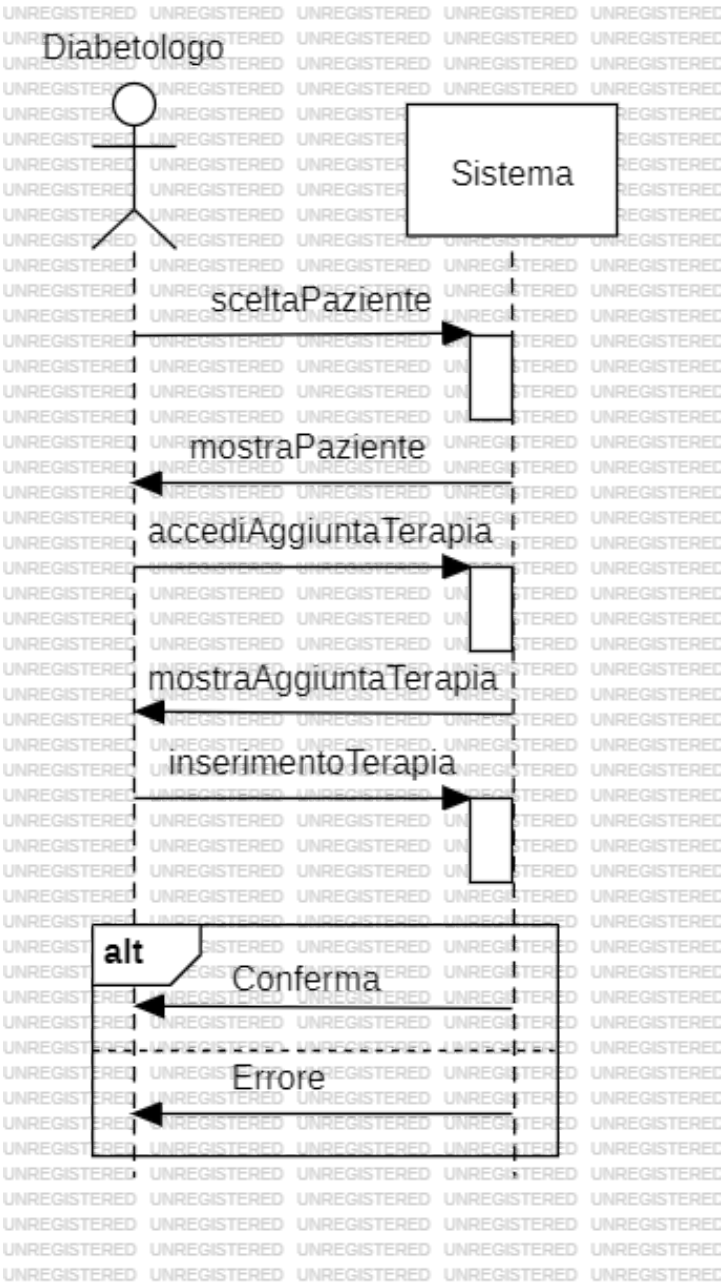

Figure 5: Aggiunta terapia

MODIFICARE O ELIMINARE UNA TERAPIA

Un diabetologo può modificare o eliminare una terapia.

Attori: Diabetologo

Precondizioni: Il diabetologo è autenticato e ha dei pazienti assegnati

Passi:

1. Il diabetologo accede al sistema
2. Il diabetologo è introdotto all'interfaccia di base
3. Il diabetologo sceglie il paziente in cura
4. Il diabetologo accede all'interfaccia per la modifica di una terapia
5. Il diabetologo modifica o elimina

Postcondizioni: L'aggiunta o la modifica della terapia è aggiunta al sistema

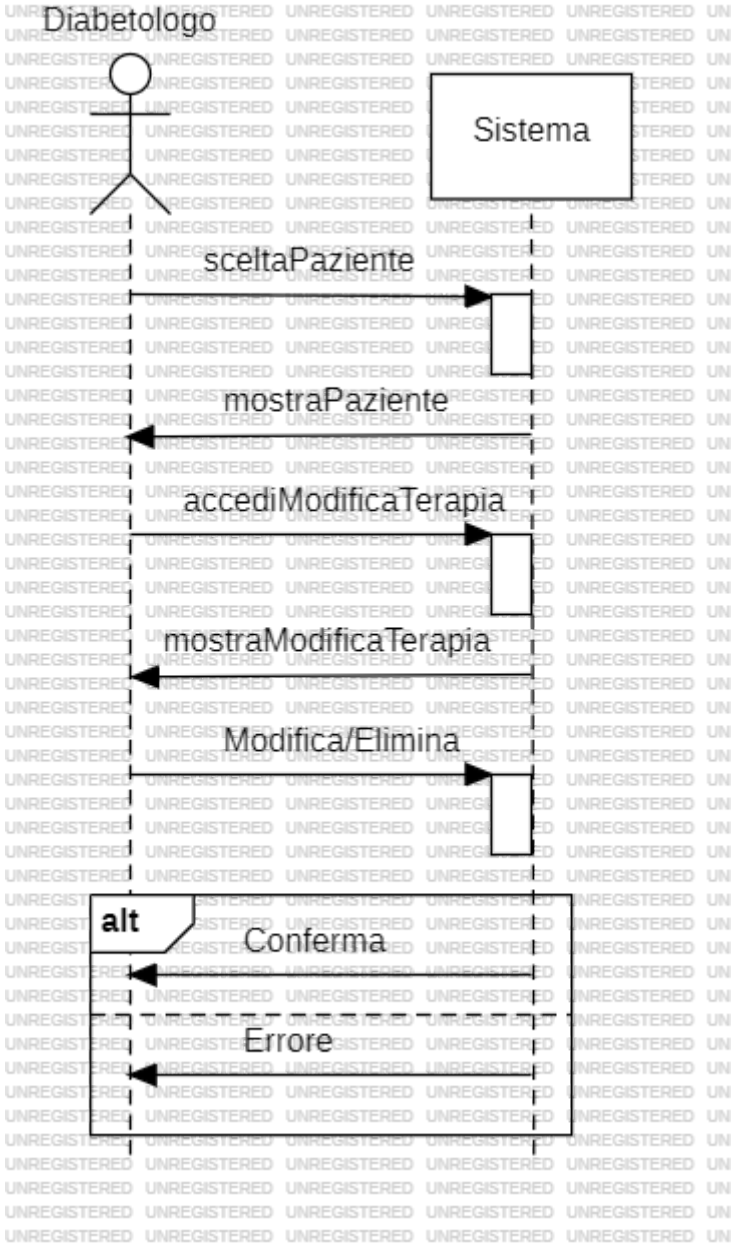

Figure 6: Modifica terapia

VISUALIZZARE I DATI DEI PAZIENTI

Un diabetologo può visualizzare i dati di un suo paziente, in particolare: le rilevazioni della glicemia del paziente, l'andamento dei valori glicemici e i sintomi segnalati o i farmaci assunti.

Attori: Diabetologo

Precondizioni: Il diabetologo è autenticato e ha dei pazienti assegnati

Passi:

1. Il diabetologo accede al sistema
2. Il diabetologo è introdotto all'interfaccia di base
3. Il diabetologo sceglie il paziente in cura
4. Il diabetologo accede all'interfaccia per la visualizzazione dei dati
5. Il diabetologo visualizza i dati del paziente

Postcondizioni: Nessuna

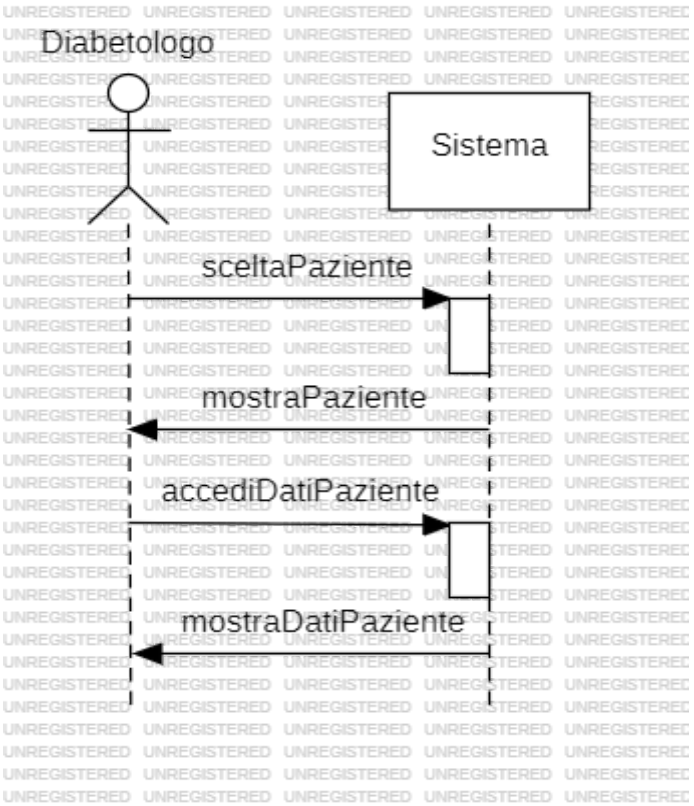

Figure 7: Visualizza dati paziente

AGGIORNAMENTO DELLE INFORMAZIONI DEL PAZIENTE

Un diabetologo può aggiornare le informazioni sul paziente con note o segnalazioni.

Attori: Diabetologo

Precondizioni: Il diabetologo è autenticato e ha dei pazienti assegnati

Passi:

1. Il diabetologo accede al sistema
2. Il diabetologo è introdotto all'interfaccia di base
3. Il diabetologo sceglie il paziente in cura
4. Il diabetologo accede all'interfaccia per la modifica dei dati
5. Il diabetologo aggiorna i dati del paziente

Postcondizioni: L'aggiornamento dei dati del paziente è aggiunta al sistema

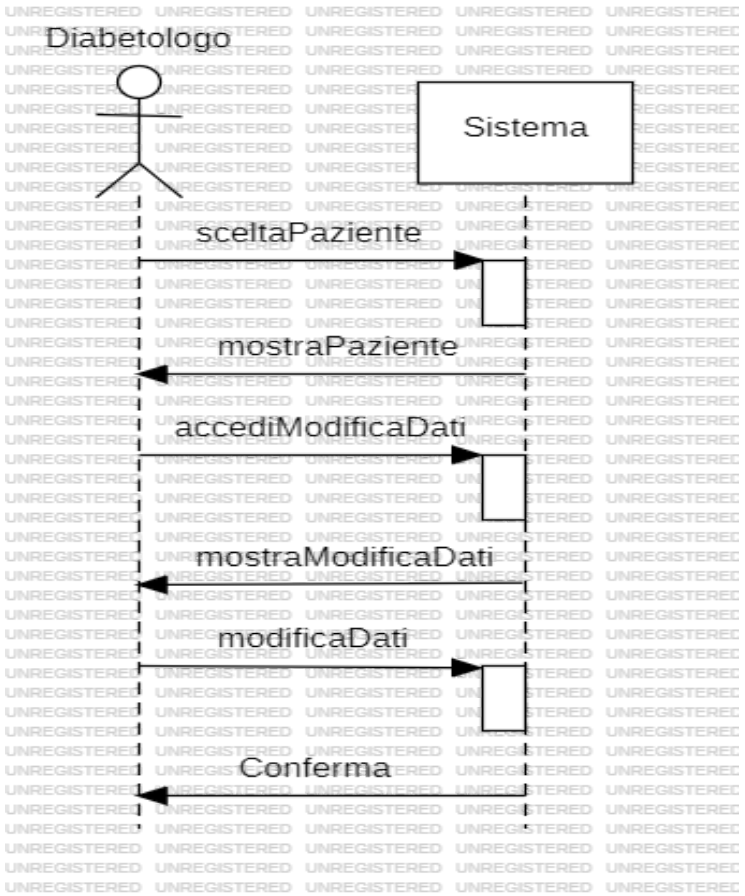

Figure 8: Aggiorna dati paziente

Diagrammi di attività

Nota: *i seguenti diagrammi rappresentano una singola attività di un utente rispetto al sistema. Non è stata rappresentata nel diagramma la possibilità di ripetere più volte la stessa operazione. Questo per ragioni di chiarezza e leggerezza.*

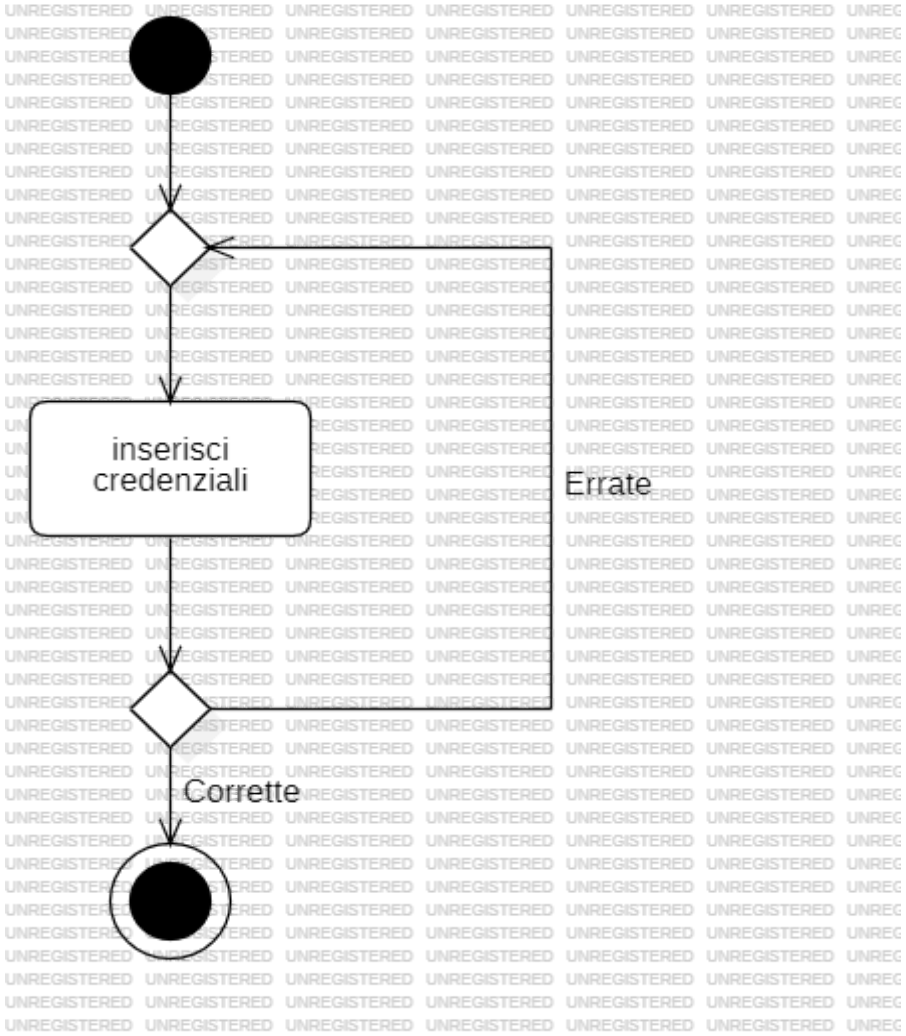

Figure 9: Autenticazione

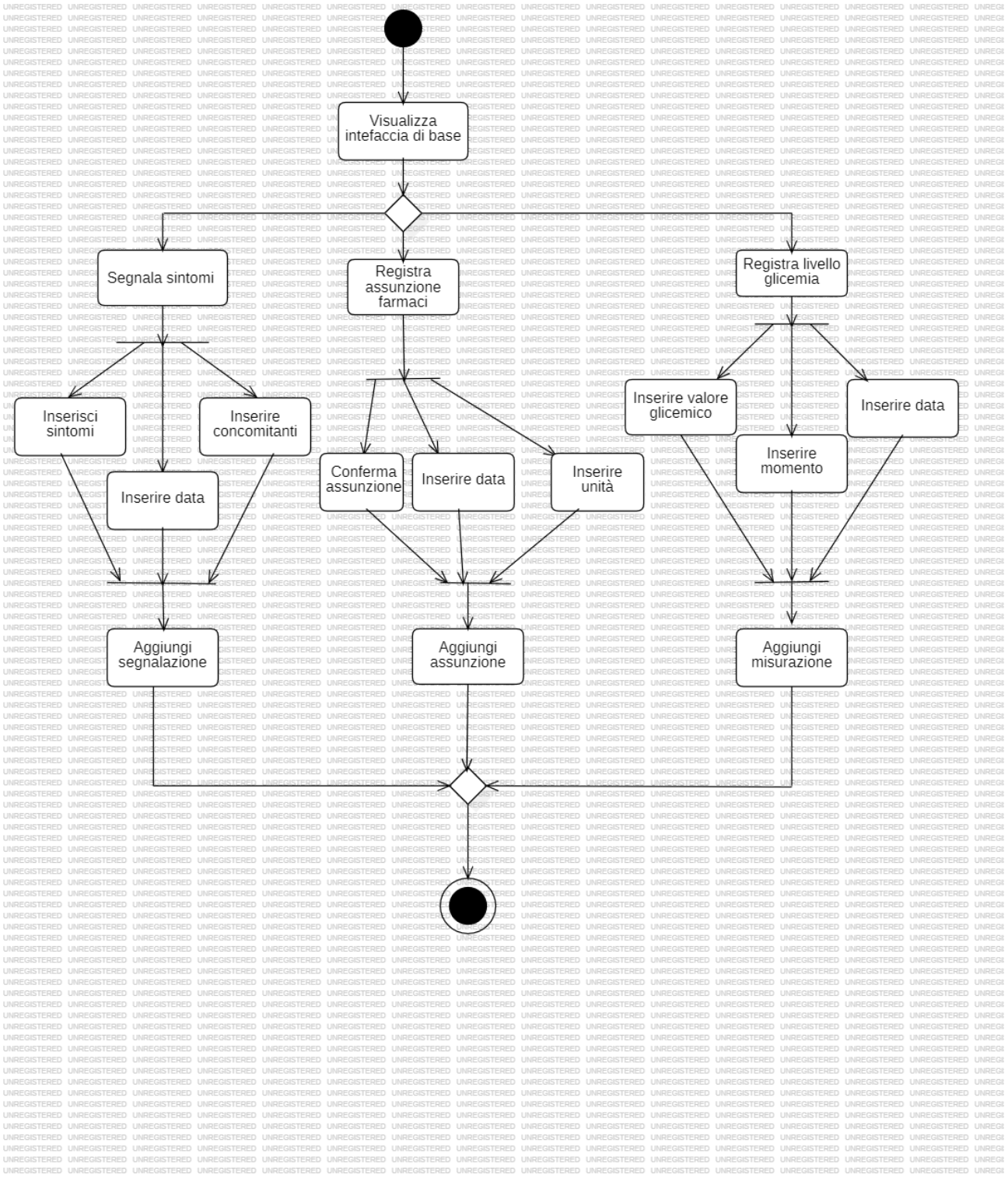

Figure 10: Attività paziente

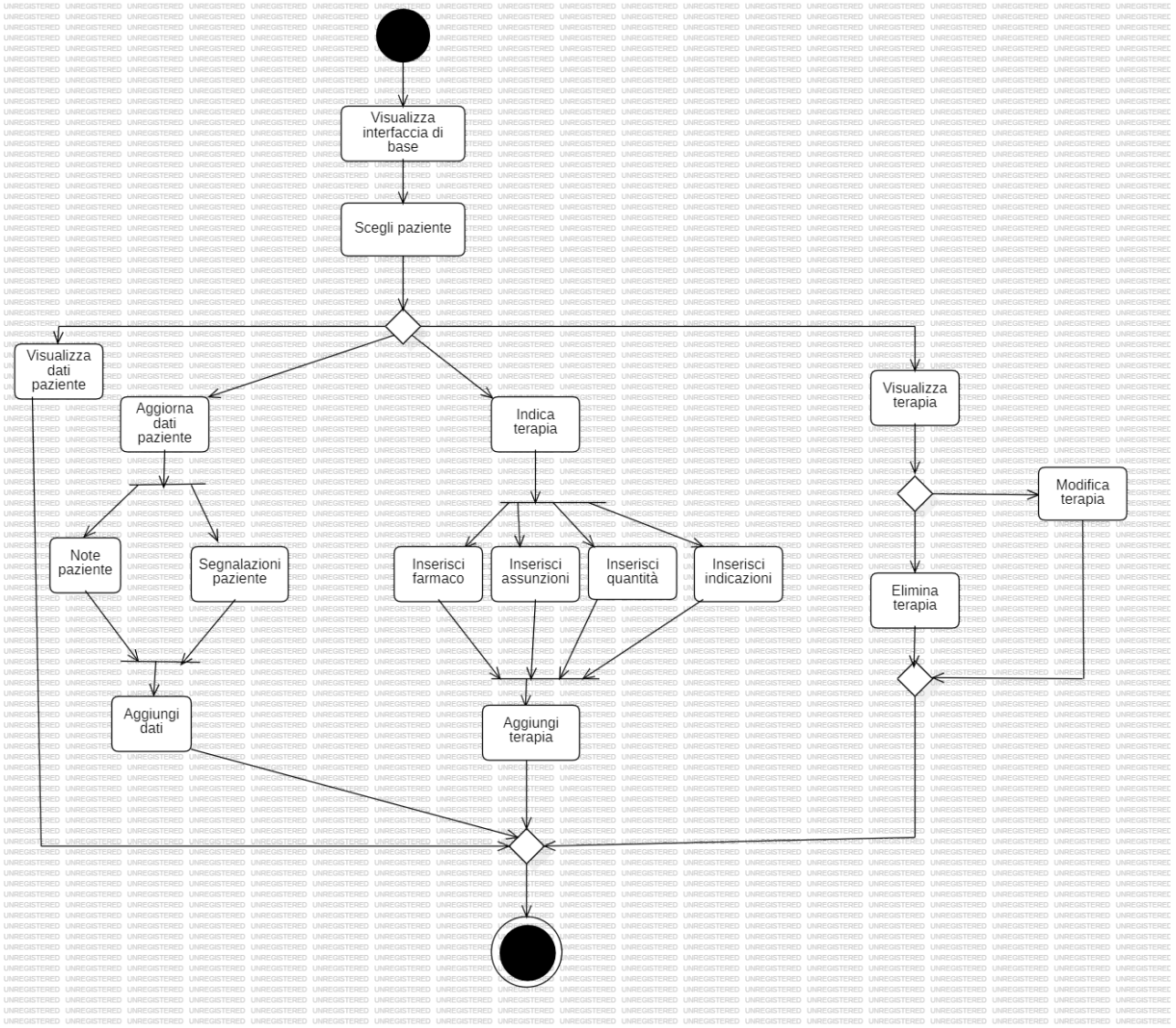

Figure 11: Attività diabetologo

Sviluppo e progettazione

Note sul processo di sviluppo

Lo sviluppo del sistema è stato condotto seguendo un approccio agile e incrementale, mantenendo comunque una certa linearità tra le fasi di analisi, progettazione, realizzazione e verifica. In primo luogo è stata svolta l'analisi dei requisiti, utile a individuare gli elementi fondamentali, i soggetti coinvolti e le loro interazioni. Successivamente si è preferito dare priorità alla progettazione, così da avere un modello di riferimento stabile prima di procedere con l'implementazione. L'attività di codifica non è stata suddivisa in compiti rigidi o in piani prestabiliti, ma è avanzata in maniera incrementale, arricchendo progressivamente il sistema con nuove funzionalità. Durante tutto il processo sono stati eseguiti test continui per garantire l'affidabilità delle soluzioni sviluppate. Inoltre, le decisioni di design e le fasi di programmazione sono state svolte in collaborazione attraverso tecniche di pair designing e pair programming, favorendo confronto costante, condivisione di idee e revisione reciproca del codice.

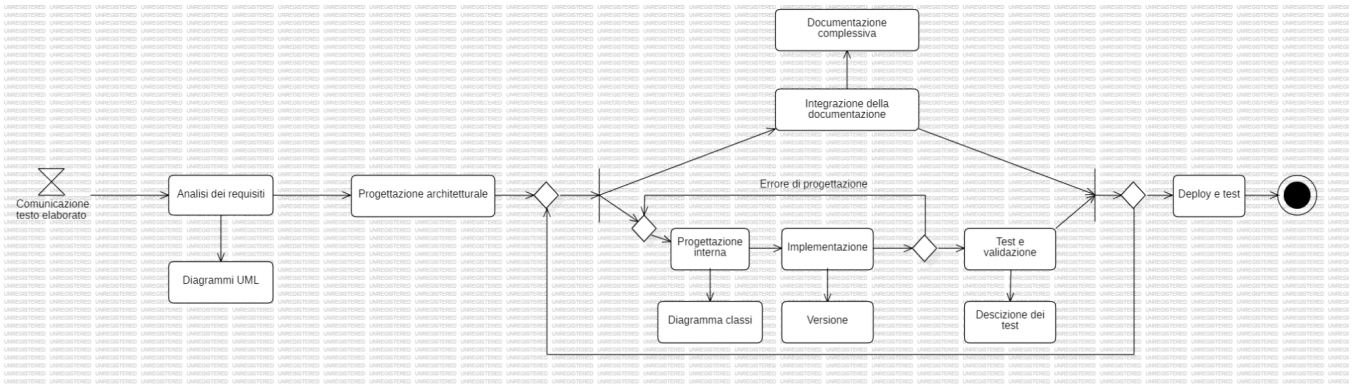

Figura 12: Processo di sviluppo

Progettazione e pattern architetturali

Il sistema è stato concepito e realizzato seguendo un approccio di modellazione ad oggetti, al fine di garantire una progettazione chiara e strutturata delle entità e delle loro relazioni. Dal punto di vista architetturale si è scelto di adottare il pattern Model–View–Controller (MVC), che ha consentito di mantenere una netta separazione tra lo strato di presentazione, la logica applicativa e la gestione dei dati. Tale scelta ha permesso la separazione delle tre componenti a livello logico e di sviluppo, dove ogni membro è stato inserito in un package separato.

- **Modello**: sottoparte del sistema che riguarda i dati e le informazioni memorizzate. Definisce la struttura dell'informazione gestita dal sistema.
- **Vista**: sottoparte del sistema che rappresenta visivamente il modello.
- **Controllore**: sottoparte del sistema che definisce la logica applicativa, ovvero il comportamento del sistema a fronte degli stimoli esterni.

Le azioni dell'utente vengono catturate dai listener, che sono costruiti per reagire di conseguenza, modificando poi le informazioni contenute nel Modello e quindi andando ad aggiornare la Vista. Non sono state fatte ulteriori articolazioni: modello, vista e controllore sono tre blocchi divisi.

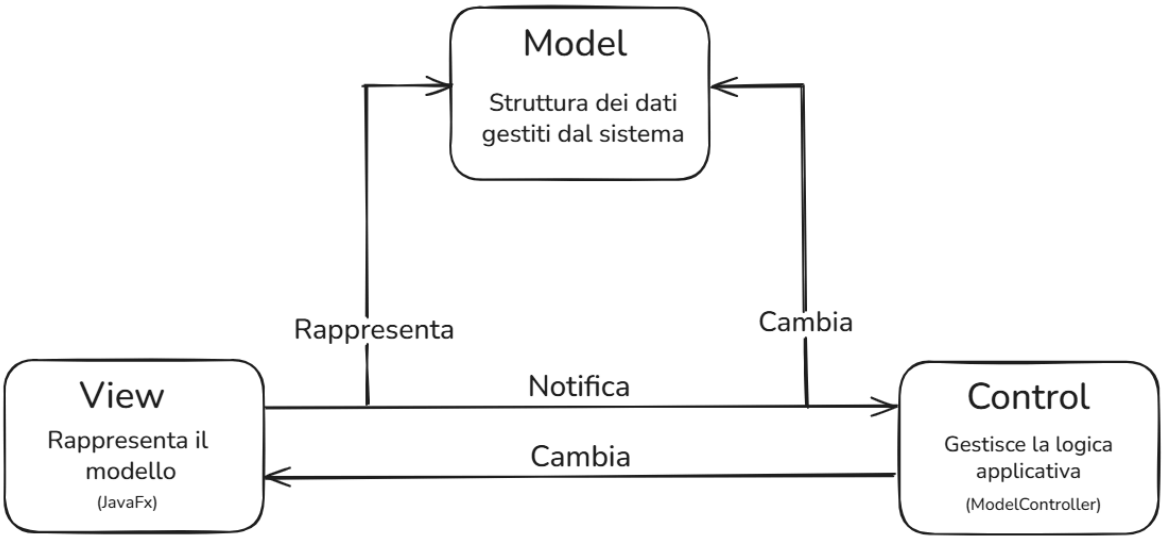

Figura 13: Architettura MVC

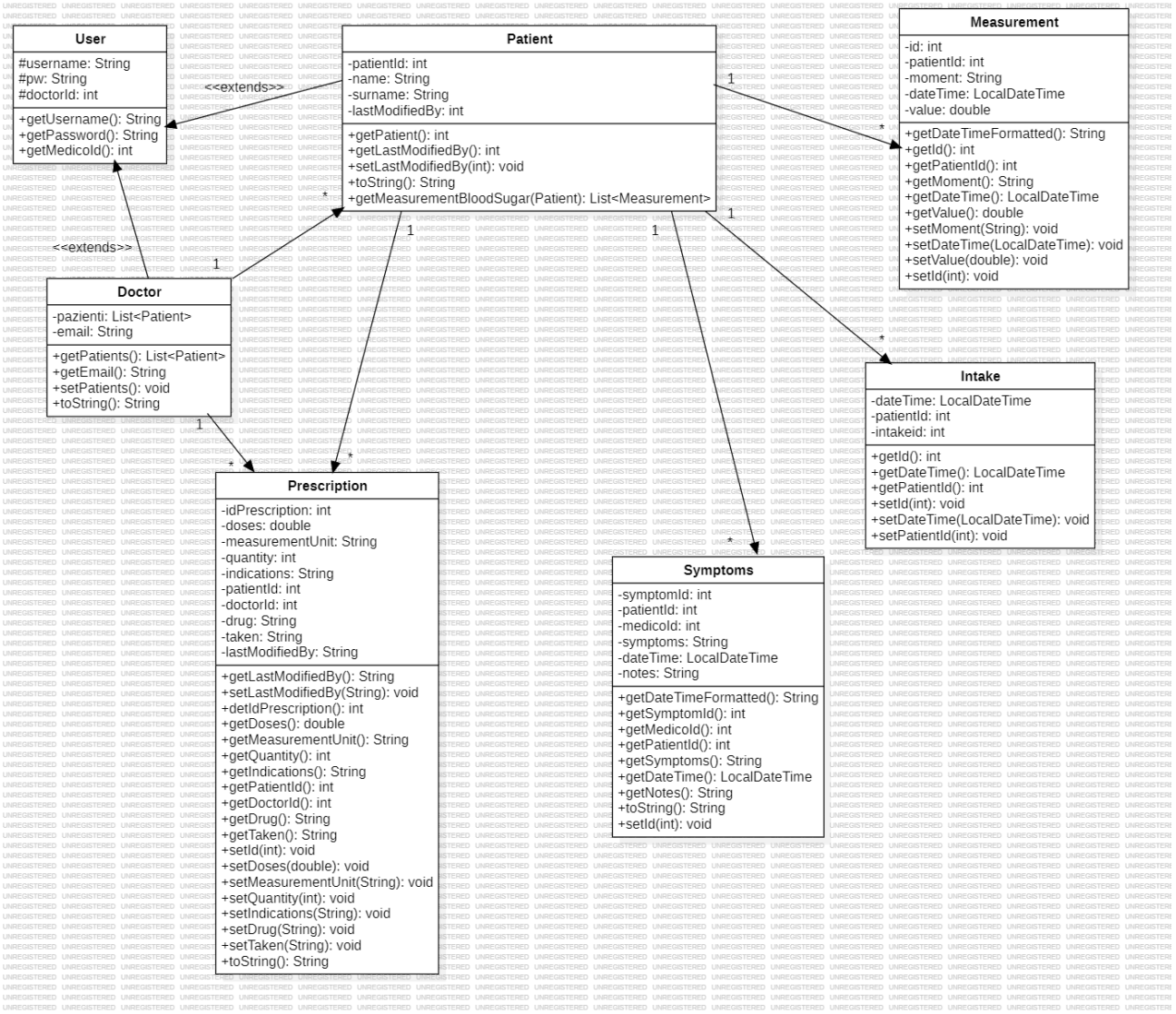

Figura 14: Diagramma delle classi del modello

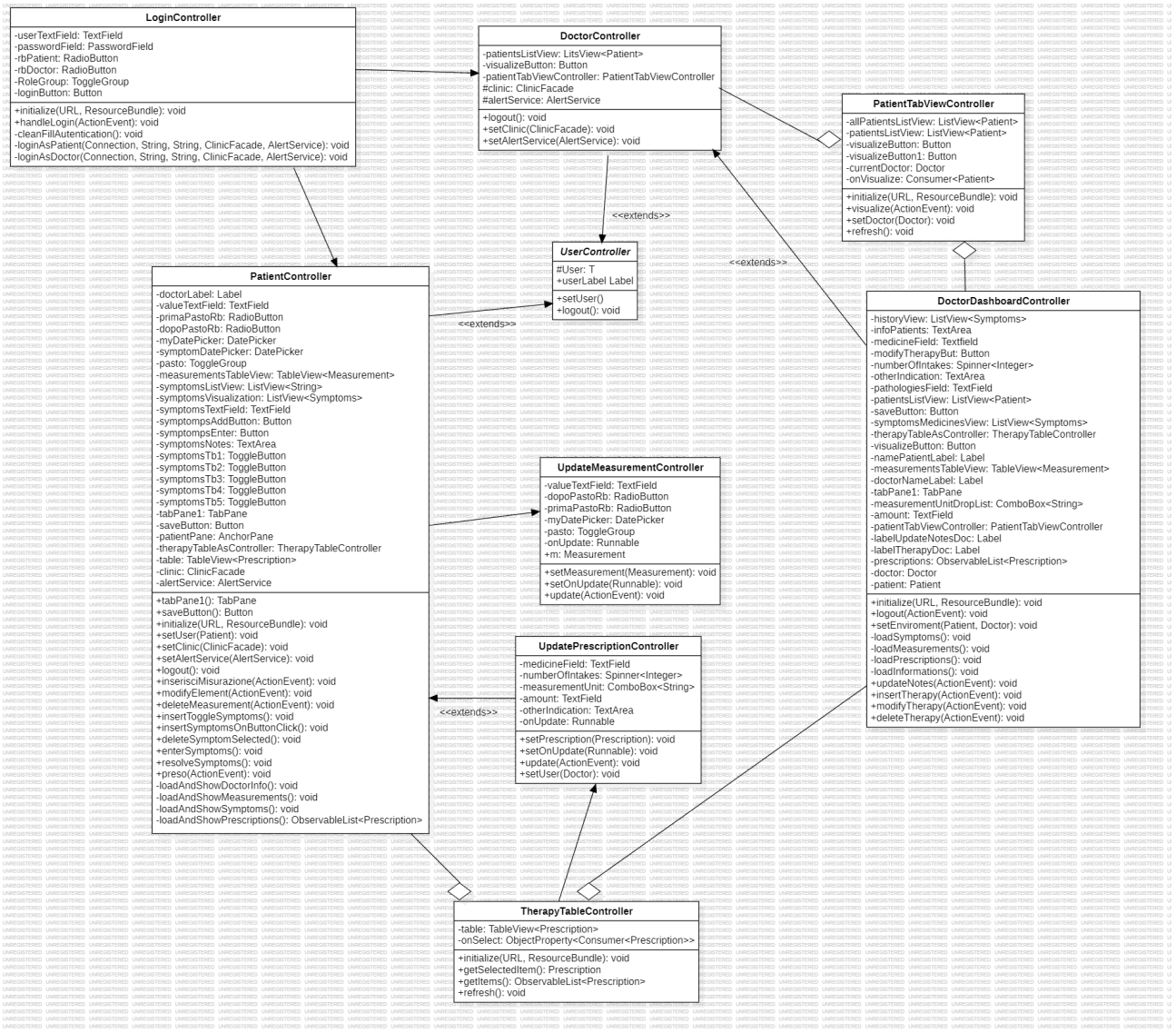

Figure 15: Diagramma delle classi dei Controller

Implementazione e design pattern

Dao

Lo strato di persistenza è stato organizzato mediante l'applicazione del pattern Data Access Object (DAO). Per ciascuna entità principale del dominio applicativo (Patient, Prescription, Measurement, Intake, Symptom) è stato sviluppato un DAO specifico, responsabile delle operazioni di inserimento, modifica, eliminazione e recupero dei dati. In questo modo le query SQL e la logica di accesso al database sono state isolate in un livello dedicato, riducendo le dipendenze e migliorando la coesione interna dei moduli. Ciò rende inoltre agevole l'eventuale sostituzione del motore di persistenza senza impatti significativi sul resto del sistema.

Facade

A livello di logica applicativa è stato introdotto il pattern Facade, con la realizzazione di due componenti principali: Clinic Facade e Alert Service.

La Clinica Facade si configura come punto di accesso unificato per le funzionalità cliniche generali, coordinando le chiamate ai vari DAO e semplificando l'interazione tra i controller e lo strato di persistenza. In questo modo i controller sono sollevati dalla necessità di dialogare direttamente con le singole componenti sottostanti, beneficiando invece di un'interfaccia coerente e centralizzata. Ciò riduce la complessità del codice, limita l'accoppiamento e rende l'applicazione più manutenibile ed estendibile.

L'Alert Service, invece, rappresenta un Facade specializzato nella gestione degli avvisi clinici. Questo modulo accentra la logica di generazione, monitoraggio e validazione degli alert, garantendo un controllo uniforme e affidabile delle condizioni critiche relative ai pazienti, ed evitando la frammentazione del codice in più parti del sistema.

L'adozione del pattern Facade nel nostro progetto si è rivelata particolarmente utile per vari motivi:

- ***Riduzione della complessità:*** i controller (es. Patient Controller, Doctor Controller) devono gestire scenari eterogenei, come inserimento di misurazioni, prescrizioni, assunzioni, sintomi e alert. Grazie alle facciate, tutte queste operazioni sono esposte tramite metodi semplici e ben definiti, senza la necessità di conoscere i dettagli interni dei relativi DAO.

- ***Chiarezza architetturale:*** separando il livello di interazione con il database (DAO) dallo strato logico dei controller, le responsabilità risultano meglio distribuite e rispettano il principio di single responsibility.
- ***Facilità di manutenzione ed estensione:*** eventuali modifiche al database o ai DAO possono essere gestite all'interno delle Facade, senza impattare direttamente sul codice dei controller e delle view. Questo rende più semplice introdurre nuove funzionalità (ad esempio un nuovo tipo di alert o una diversa logica di prescrizione).
- ***Riutilizzabilità e coerenza:*** l'uso di un'interfaccia unificata permette di utilizzare la logica clinica in più punti dell'applicazione (sia lato medico che lato paziente) mantenendo coerenza nei risultati.
- ***Maggiore testabilità:*** concentrando la logica in pochi punti di accesso, diventa più semplice scrivere test mirati (unit test e integration test) per validare la correttezza delle operazioni complesse.

L'adozione congiunta dei pattern MVC, DAO e Facade ha permesso di ottenere un'architettura modulare, scalabile e ben stratificata. Ogni livello risulta così indipendente ma al contempo perfettamente integrato con gli altri, favorendo lo sviluppo incrementale e la collaborazione all'interno del team. L'impiego di tali soluzioni architetturali non solo ha migliorato la qualità del codice e la chiarezza del progetto, ma ha anche consentito di predisporre una base solida per eventuali evoluzioni e ampliamenti futuri del sistema.

Attività di test e validazione

Per assicurare la solidità e l'affidabilità del sistema sono state condotte diverse attività di verifica e validazione, articolate su più livelli:

- ***Ricognizione delle specifiche*:** è stato effettuato un confronto puntuale tra il documento dei requisiti e i diagrammi UML prodotti (casi d'uso, diagrammi di classe e di sequenza), al fine di garantire la completa copertura funzionale e l'assenza di omissioni.
- ***Verifica della consistenza tra modelli e implementazione*:** si è controllata la corrispondenza tra i diagrammi progettuali e il codice sorgente sviluppato, assicurandosi che le strutture implementate rispettassero le astrazioni definite in fase di analisi e progettazione.
- ***Ispezione del codice*:** il codice è stato sottoposto a revisione per individuare eventuali anomalie, verificare la corretta applicazione dei design pattern previsti (MVC, DAO, Facade, ecc.) e rilevare possibili bad practices (duplicazioni, accoppiamento eccessivo, mancanza di incapsulamento).

Per quanto riguarda il test automatizzato, si è scelto di utilizzare **JUnit**, concentrandosi principalmente sui **DAO** (Data Access Object). Tale scelta è stata dettata dal fatto che rappresentano lo strato più critico del sistema, in quanto responsabili del collegamento diretto con il database e quindi garanti della **persistenza** e della **consistenza dei dati clinici** (pazienti, medici, prescrizioni, misurazioni, assunzioni, sintomi e avvisi).

Per quanto riguarda invece le componenti di *interfaccia grafica*, si è preferito adottare un approccio manuale: le **view** sono state testate direttamente attraverso l'utilizzo del software, osservando che il comportamento dell'applicazione fosse coerente con i requisiti e con gli scenari d'uso previsti.

Successivamente, sono state condotte due ulteriori tipologie di test a livello applicativo:

- *Test degli sviluppatori*: esecuzione manuale di scenari d'uso con input corretti e scorretti, database vuoto o popolato, verifiche sugli alert e sul tracciamento

delle operazioni eseguite dai medici. Questi test hanno permesso di validare la logica complessiva e la gestione delle eccezioni.

- ➢ Test utente generico: il sistema è stato sottoposto a prove da parte di utilizzatori esterni, privi di competenze informatiche specifiche e non coinvolti nello sviluppo. L'obiettivo è stato osservare la naturale interazione con l'interfaccia, individuare errori non previsti e raccogliere spunti per il miglioramento dell'usabilità complessiva.

Unit test

Dal momento che gran parte dell'applicazione si fonda sull'interazione con il database, si è ritenuto opportuno implementare test di unità — tramite JUnit — per verificare il corretto funzionamento delle classi DAO, ossia le componenti che incapsulano l'accesso ai dati.

In particolare, sono state sottoposte a test le seguenti classi:

1. JdbcPatientDao: responsabile di tutte le operazioni sul database relative ai pazienti;
2. JdbcMeasurementDao: responsabile di tutte le operazioni sul database relative alle misurazioni cliniche.

La scelta di limitare i test a queste due classi non è casuale: esse rappresentano i casi più significativi, in quanto coprono l'insieme principale delle operazioni tipiche sui dati (inserimento, modifica, cancellazione e recupero). Le altre implementazioni dei DAO presentano logiche pressoché analoghe, differenziandosi soltanto per l'entità gestita (ad esempio prescrizioni, sintomi, assunzioni di farmaci). Per questo motivo, testare anche quelle avrebbe comportato una mera ripetizione degli stessi controlli, senza aggiungere reale valore alla validazione complessiva del sistema.

In sintesi, il testing mirato su JdbcPatientDao e JdbcMeasurementDao consente di garantire un adeguato livello di affidabilità dell'interazione con il database, evitando ridondanza nei test e mantenendo efficiente il processo di verifica

Test degli sviluppatori

In questa fase gli sviluppatori hanno immesso nel sistema vari input (corretti e scorretti) con lo scopo di verificare che le funzionalità implementate rispondessero ai

requisiti. I test hanno riguardato sia aspetti di interfaccia utente che di consistenza dei dati a livello di database e logica applicativa.

Le principali prove condotte sono state:

- ***Verifica del corretto funzionamento dell'autenticazione***: inserendo credenziali errate l'accesso viene negato, mentre con credenziali corrette l'utente viene indirizzato alla schermata iniziale associata al suo ruolo (paziente o medico).

- ***Medico a database vuoto:*** testando l'interfaccia senza pazienti, prescrizioni o misurazioni, è stato verificato che i pulsanti e le funzionalità non generassero eccezioni o comportamenti anomali.

- ***Paziente a database vuoto:*** analogamente, si è verificato che in assenza di dati clinici la navigazione fosse possibile senza errori e che venissero mostrate viste vuote o messaggi informativi invece di crash.

- ***Inserimento di nuove misurazioni:*** controllata la corretta registrazione delle misurazioni glicemiche, la coerenza dei valori (non negativi o malformati), e la loro visualizzazione sia da parte del paziente sia da parte di tutti i medici del sistema, come previsto dai requisiti.

- ***Controllo degli input anomali:*** sono stati immessi dati incompleti (campi vuoti), stringhe non valide, valori negativi. Il sistema ha correttamente respinto o segnalato tali input

- ***Inserimento di assunzioni farmacologiche:*** È stata verificata la corretta associazione di ciascuna assunzione alla prescrizione corrispondente, garantendo che nella view del paziente vengano mostrate esclusivamente le terapie a lui assegnate, senza possibilità di visualizzare o interagire con prescrizioni relative ad altri pazienti. Contestualmente, il sistema effettua l'aggiornamento dei campi giornalieri della prescrizione, marcando la dose prevista come "assunta" nel momento in cui il paziente conferma l'assunzione. Il paziente ha comunque tempo fino alla fine della giornata per modificare l'informazione, qualora avesse segnalato erroneamente un'assunzione mai avvenuta. L'effettiva verifica avviene al login del giorno successivo: se il

sistema rileva che il giorno precedente non è stata registrata alcuna assunzione, viene salvata nel database la data e il nominativo del paziente che ha mancato la terapia giornaliera.

Questo meccanismo consente di monitorare in maniera accurata l'aderenza terapeutica: qualora vengano rilevate tre giornate consecutive di mancata assunzione, il sistema genera automaticamente un alert visibile al medico, che potrà intervenire tempestivamente per valutare la situazione clinica del paziente.

La scelta di richiedere che tutti i pazienti effettuino almeno un login giornaliero è stata ritenuta ragionevole, poiché l'utilizzo quotidiano del software è parte integrante del percorso terapeutico: i pazienti devono infatti inserire regolarmente le misurazioni glicemiche, registrare le assunzioni dei farmaci e tenere aggiornato il proprio quadro clinico. Tale impostazione rafforza il ruolo del sistema come strumento di monitoraggio continuo, migliorando sia la qualità dei dati raccolti che l'efficacia del supporto medico.

- ***Verifica del sistema di alert:*** controllato che il sistema generi avvisi in diversi casi (soglie critiche di glicemia, mancata assunzione di farmaci, promemoria periodici) e che questi siano visibili ai medici. Sono stati provati scenari con valori al limite delle soglie, valori estremi e condizioni di borderline.

- ***Accesso dei medici ai pazienti:*** è stato verificato che ogni medico possa visualizzare e modificare i dati clinici di tutti i pazienti presenti nel sistema, in conformità con i requisiti, garantendo quindi un accesso globale alle cartelle cliniche.

- ***Tracciamento delle operazioni:*** è stato introdotto un sistema di logging a livello applicativo che registra quale medico ha effettuato modifiche ai dati (aggiornamento di terapie, aggiornamento della cartella clinica). Si è verificato che la label di tracciamento presente nell'interfaccia si aggiorni correttamente, mostrando il nome del medico autore dell'operazione in tempo reale. Questo consente una maggiore trasparenza e tracciabilità delle modifiche.

Test utente generico

Come ultimo scaglione il software è stato sottoposto a un test da parte di alcuni individui con limitata dimestichezza informatica ed assoluto distacco dallo sviluppo. In questa fase non si è cercato in alcun modo di guidare o strutturare l'esperienza, per non influenzare il risultato; piuttosto si è lasciato che il soggetto navigasse liberamente il sistema. Non è stata data nessuna spiegazione sull'utilizzo del software, se non una generale indicazione dei suoi fini; ci si è limitati a rispondere alle domande, quando sollevate. Lo scopo del test era rilevare errori invisibili allo sviluppatore; in realtà, gli utenti a cui è stato mostrato il software hanno anche contribuito a individuare nuove funzionalità utili a migliorare l'usabilità generale del sistema.